

Suraksha

Architecture & Methodology — on-device scam defence for rural India (The Maverick Effect AI Challenge 2026)

1. Design principles

- **Model the psychology, not the wording.** Scam text mutates; the manipulation grammar does not.
- **No single model owns the decision.** Three components are fused, so one blind spot is caught by others.
- **Everything is explainable.** Every verdict names the phrases, the tactic, and the archetype behind it.
- **On-device first.** All inference runs in the browser: zero marginal compute, no PII leaves the phone.
- **Safety-first when uncertain.** A missed scam costs savings; a false alarm costs ten seconds.
- **Honest engineering.** We never claim what the browser cannot do (e.g. tapping a live call).

2. The three components

A. Neural classifier (semantic). paraphrase-multilingual-MiniLM-L12-v2 (384-dim) runs in-browser as an int8 ONNX build (Transformers.js, WebGPU->WASM). A logistic-regression head over [embedding(384), rule-groups(8), meta(6)] outputs pScam in [0,1]. It is lazily loaded on first check and degrades to rules-only if absent.

B. Deterministic rule engine (structural). Pure TypeScript, 8 groups: credential, UPI, URL, loan, impersonation, urgency, reward, textual. Each rule emits a weighted signal with a translated reason and the exact evidence span. APK/EXE links and explicit OTP/PIN asks carry the highest weight.

C. Tactic scorer + archetype matcher. Scores 8 axes 0-100 (Urgency, Authority, Fear, Reward, Secrecy, Credential ask, Fake trust, Irreversibility), drawn as an SVG radar, and matches 16 known Indian fraud archetypes - including what the scammer will ask for next.

3. Fusion & calibration

```
riskScore = clamp( 0.45·pScam + 0.40·ruleScore + 0.15·tacticPeak , 0, 100 )
```

- Bands: SAFE 0-24 · CAUTION 25-54 · RISKY 55-79 · DANGER 80-100.
- **Hard overrides** bypass the weights: an explicit credential ask or an APK link floors the band to DANGER regardless of pScam, and the floor survives fusion.
- Rules verdict renders in <300ms; the neural score then refines the same result shape.

4. Honest metrics

From notebooks/train.ipynb; regenerated as the corpus grows. Reported small and real rather than a vague 95%. The corpus is being expanded to a trilingual ~1,200-row target with native speakers.

Configuration	F1	Recall	Precision
Rules only	0.786	0.786	0.786
Metadata only	0.800	0.857	0.750
Embeddings only	0.815	0.786	0.846

Embeddings + metadata	0.857	0.857	0.857
Full ensemble	0.923	0.857	1.000

- 5-fold CV F1 mean **0.844**.
- Ablation: rules-only F1 **0.786** -> **0.923** full ensemble, lifting precision **0.786** -> **1.000** while holding recall - the embedding layer mainly removes false alarms.
- Threshold tuned for **recall >= 0.95**: we optimise for catching scams and say so.

5. Data, quality & reproducibility

- Corpus data/corpus.jsonl {id, text, lang, label, archetype, tactics[], source}, published in the repo; real legitimate messages included so the model does not flag everything.
- Training in a reproducible Colab notebook: embed -> train head -> evaluate -> export JSON.
- Cross-runtime parity check so Python-trained and browser-scored embeddings agree.
- TypeScript strict (noUncheckedIndexedAccess); **405 unit tests** (TDD); a test fails if any reason is missing in any language; a lint rule reserves verdict colours for verdict UI only.

6. Performance, offline & portability

- ~30MB int8 model lazy-loaded once, then cached by a service worker; installable PWA, works fully offline.
- Engine is pure, framework-free TypeScript - portable to an SMS/IVR fallback, a bank/NGO widget, or a WhatsApp helper. No bank/NPCI integration today; described as a future path, never faked.